



CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
ESCOLA SUPERIOR POLITÉCNICA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DESENVOLVIMENTO WEB - BACK END

ATIVIDADE PRÁTICA
API REST – BAOZI STORE

DIEGO MANAGLIA – RU: 5565650

OSASCO – SP
2026

ATIVIDADE PRÁTICA

1. Descrição de uma situação fictícia

A Baozi Store é uma pequena loja que vende pão chinês (baozi). Para melhorar a organização do negócio, foi criado um sistema simples para controlar clientes, produtos e pedidos.

Um cliente chamado **DiegoManaglia5565650** realizou seu cadastro no sistema em 09/10/2026. O produto vendido pela loja chama-se **Baozi de Frango**, um pão chinês recheado com frango, vendido por unidade ao preço de **R\$ 12,99**.

Em um determinado momento, o cliente realizou um pedido de **6 unidades** do produto. O sistema registra o cliente, o produto comprado e a quantidade solicitada, facilitando o controle da loja.

Os dados dessa situação foram cadastrados na API e aparecem nos prints do Postman da seção 4:

Entidade	Dados cadastrados
Cliente	ID 1 – nome DiegoManaglia5565650, cliente desde 09/10/2026
Produto	ID 1 – Baozi de Frango, R\$ 12,99, em estoque
Pedido	ID 1 – cliente 1 (DiegoManaglia5565650), produto 1 (Baozi de Frango), 6 unidades

2. Diagrama caso de uso

O diagrama abaixo representa as funcionalidades implementadas na API REST. O ator principal é o **Usuário da API**, que se comunica com o sistema por meio de requisições HTTP (nos testes, feitas pelo Postman).

Para cada entidade (Cliente, Produto e Pedido) existem cinco casos de uso: cadastrar, listar, consultar por ID, atualizar e excluir, totalizando 15 casos de uso.

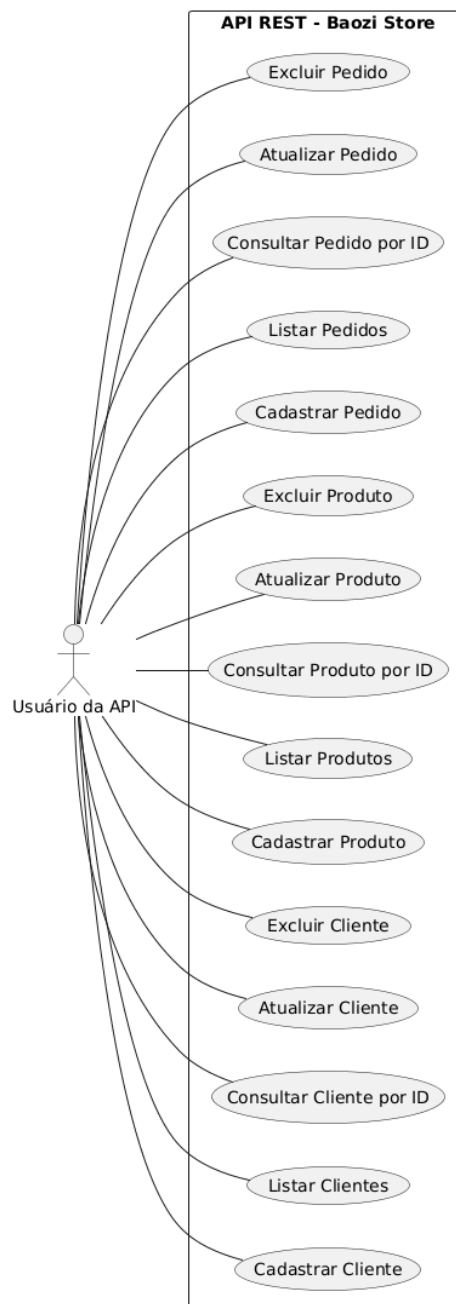


Figura 1 – Diagrama de caso de uso da API REST Baozi Store

3. Especificação da API desenvolvida

A API foi desenvolvida com **Java 21**, **Spring Boot 4.1.1**, **Spring Web**, **Spring Data JPA** e banco de dados relacional **H2** (em memória), usando o **Maven** para o build. O projeto segue o padrão MVC do Spring e está organizado nos pacotes `model` (entidades), `repository` (acesso ao banco de dados) e `controller` (endpoints REST). Todas as requisições e respostas usam o formato **JSON**.

3.1 Entidades criadas e seus campos

Entidade	Campo	Tipo	Descrição
Cliente	id	Long	Identificador gerado automaticamente
	nome	String	Nome do cliente
	clienteDesde	LocalDate	Data em que se tornou cliente
Produto	id	Long	Identificador gerado automaticamente
	nome	String	Nome do produto
	preco	BigDecimal	Preço por unidade
	estoque	Boolean	Indica se o produto está em estoque
Pedido	id	Long	Identificador gerado automaticamente
	clienteId	Long	ID do cliente que fez o pedido
	produtoId	Long	ID do produto comprado
	quantidade	Integer	Quantidade comprada

Um cliente pode fazer vários pedidos, e um produto pode ser vendido em vários pedidos. Conforme o DER do trabalho, o pedido guarda apenas o ID do cliente (`clienteId`) e o ID do produto (`produtoId`).

3.2 Endpoints implementados

A URL base da API é `http://localhost:8080`. Os endpoints abaixo foram implementados:

Método	Endpoint	Descrição	Resposta de sucesso
POST	<code>/clientes</code>	Cadastra um cliente	201 Created
GET	<code>/clientes</code>	Lista todos os clientes	200 OK
GET	<code>/clientes/{id}</code>	Consulta um cliente pelo ID	200 OK
PUT	<code>/clientes/{id}</code>	Atualiza um cliente (opcional)	200 OK
DELETE	<code>/clientes/{id}</code>	Exclui um cliente	204 No Content
POST	<code>/produtos</code>	Cadastra um produto	201 Created
GET	<code>/produtos</code>	Lista todos os produtos	200 OK
GET	<code>/produtos/{id}</code>	Consulta um produto pelo ID	200 OK
PUT	<code>/produtos/{id}</code>	Atualiza um produto (opcional)	200 OK
DELETE	<code>/produtos/{id}</code>	Exclui um produto	204 No Content

Método	Endpoint	Descrição	Resposta de sucesso
POST	/pedidos	Cadastra um pedido	201 Created
GET	/pedidos	Lista todos os pedidos	200 OK
GET	/pedidos/{id}	Consulta um pedido pelo ID	200 OK
PUT	/pedidos/{id}	Atualiza um pedido (opcional)	200 OK
DELETE	/pedidos/{id}	Exclui um pedido	204 No Content

Quando o ID informado não existe, os endpoints de consulta por ID, atualização e exclusão retornam **404 Not Found**. No cadastro e na atualização de pedidos, a API confere se o cliente e o produto informados existem; se algum não existir, retorna **400 Bad Request**.

4. Prints do Postman

Os testes foram feitos no Postman, em uma única execução da API, na ordem apresentada abaixo. Os dados usados são os mesmos da descrição da seção 1.

4.1 Criar (POST)

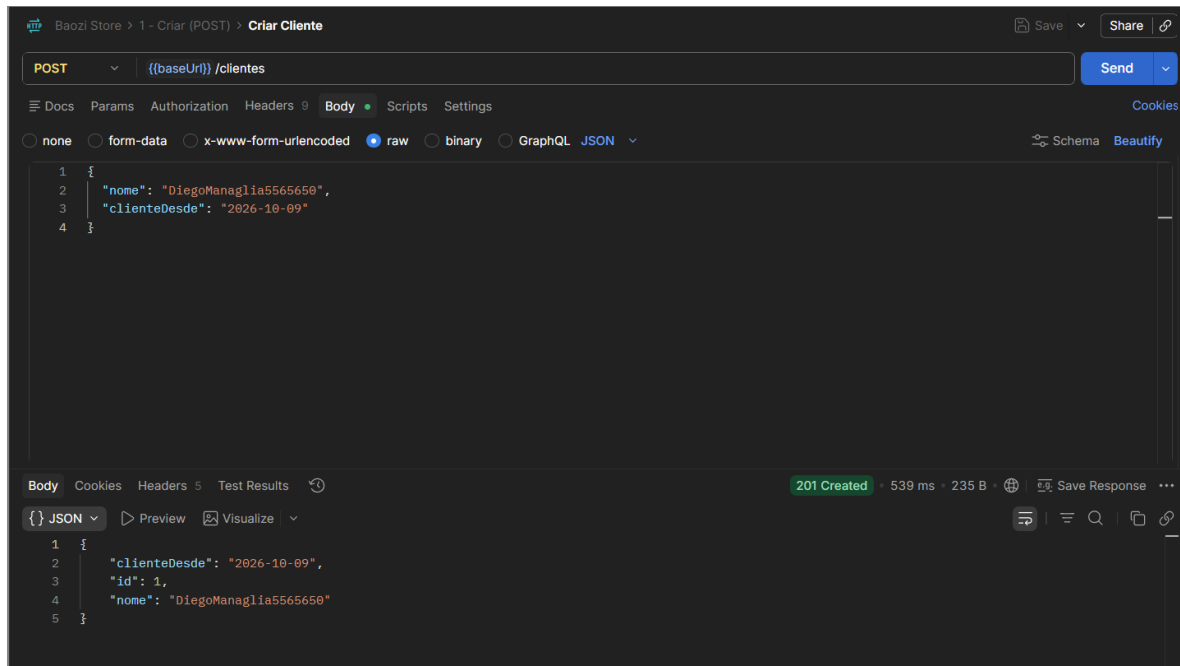


Figura 2 – POST /clientes: cadastro do cliente *DiegoManaglia5565650* (201 Created)

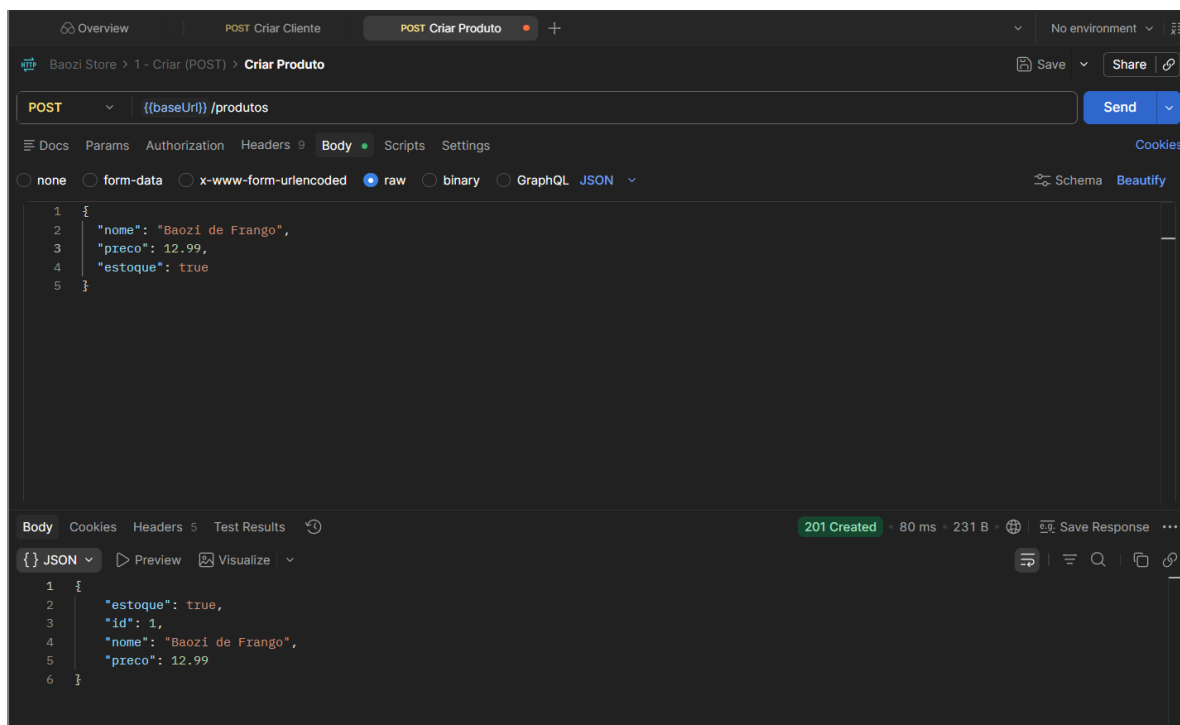


Figura 3 – POST /produtos: cadastro do produto *Baozi de Frango* (201 Created)

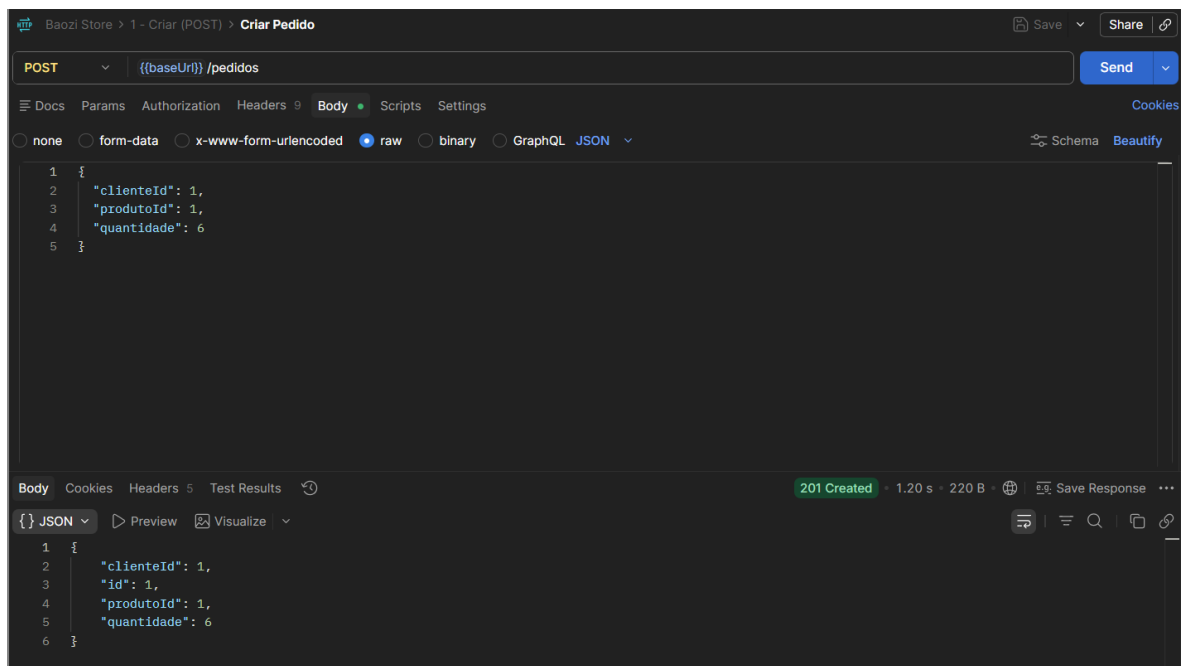


Figura 4 – POST /pedidos: cadastro do pedido de 6 unidades (201 Created)

4.2 Listagem geral (GET)

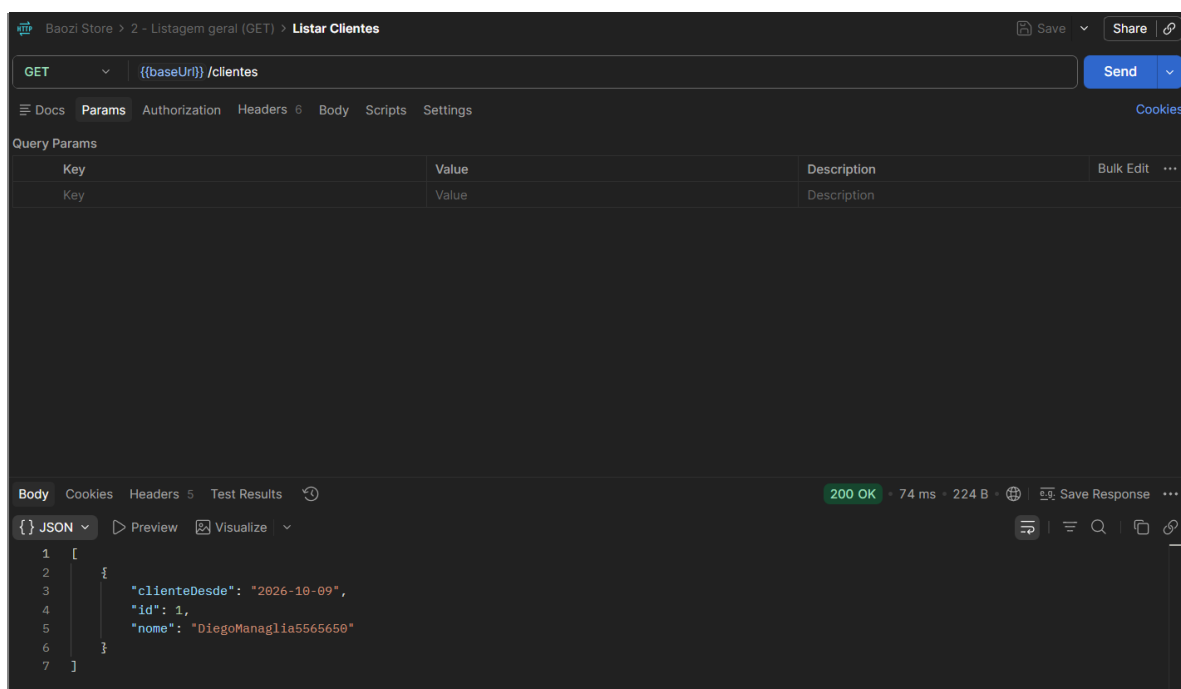


Figura 5 – GET /clientes: listagem geral de clientes (200 OK)

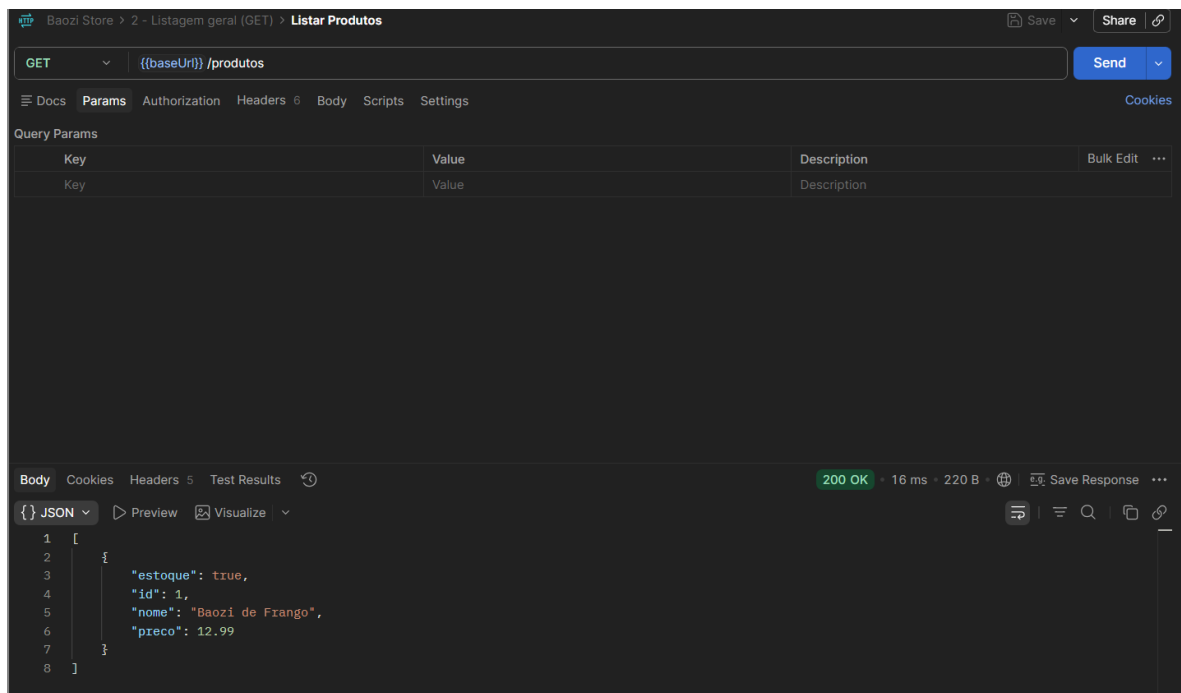


Figura 6 – GET /produtos: listagem geral de produtos (200 OK)

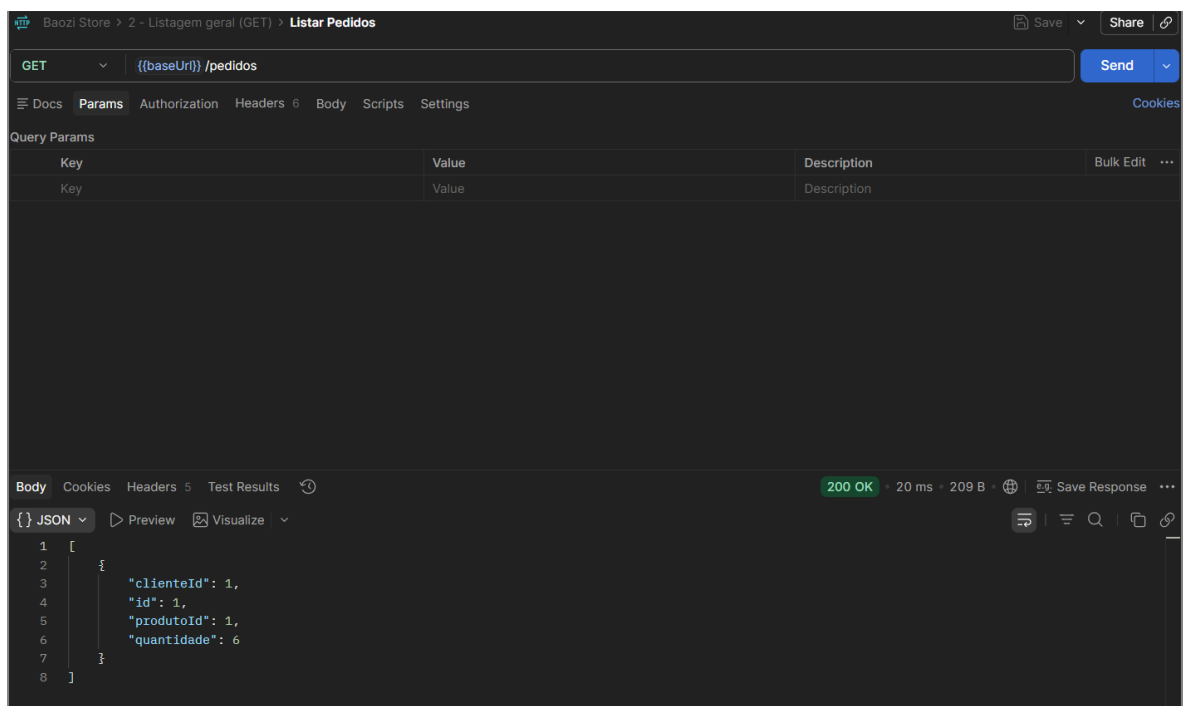


Figura 7 – GET /pedidos: listagem geral de pedidos (200 OK)

4.3 Consulta por ID (GET)

The screenshot shows a REST client interface for a project named "Baozi Store". The selected endpoint is "3 - Consulta por ID (GET)" with the sub-label "Buscar Cliente por ID". The HTTP method is "GET" and the URL is "{{baseUrl}} /clientes/1". The "Send" button is visible. Below the URL bar, there are tabs for "Docs", "Params", "Authorization", "Headers", "Body", "Scripts", and "Settings". The "Params" tab is active, showing a table for "Query Params" with columns "Key", "Value", "Description", and "Bulk Edit". The table is currently empty. The "Body" tab is also visible, showing a JSON response. The status bar at the bottom indicates a "200 OK" response with a response time of 36 ms and a body size of 230 B. The JSON response is as follows:

```
1 {
2   "clienteDesde": "2026-10-09",
3   "id": 1,
4   "nome": "DiegoManag1ia5565650"
5 }
```

Figura 8 – GET /clientes/1: consulta do cliente pelo ID (200 OK)

The screenshot shows the same REST client interface, but for a different endpoint: "3 - Consulta por ID (GET)" with the sub-label "Buscar Produto por ID". The HTTP method is "GET" and the URL is "{{baseUrl}} /produtos/1". The "Send" button is visible. The "Params" tab is active, showing an empty "Query Params" table. The "Body" tab is visible, showing a JSON response. The status bar at the bottom indicates a "200 OK" response with a response time of 16 ms and a body size of 226 B. The JSON response is as follows:

```
1 {
2   "estoque": true,
3   "id": 1,
4   "nome": "Baozi de Frango",
5   "preco": 12.99
6 }
```

Figura 9 – GET /produtos/1: consulta do produto pelo ID (200 OK)

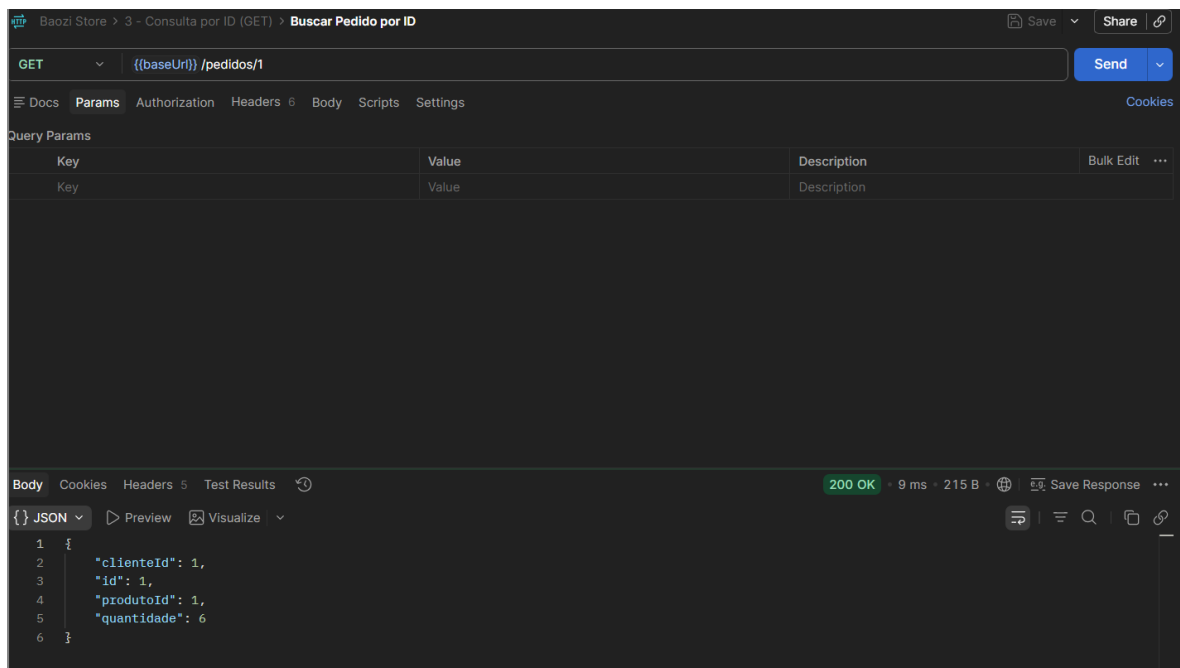


Figura 10 – GET /pedidos/1: consulta do pedido pelo ID (200 OK)

4.4 Exclusão (DELETE)

Para manter os dados da descrição, antes da exclusão foram cadastrados um cliente, um produto e um pedido extras (ID 2), que foram excluídos nos testes abaixo. Por último, a consulta do pedido excluído confirma que ele não existe mais.

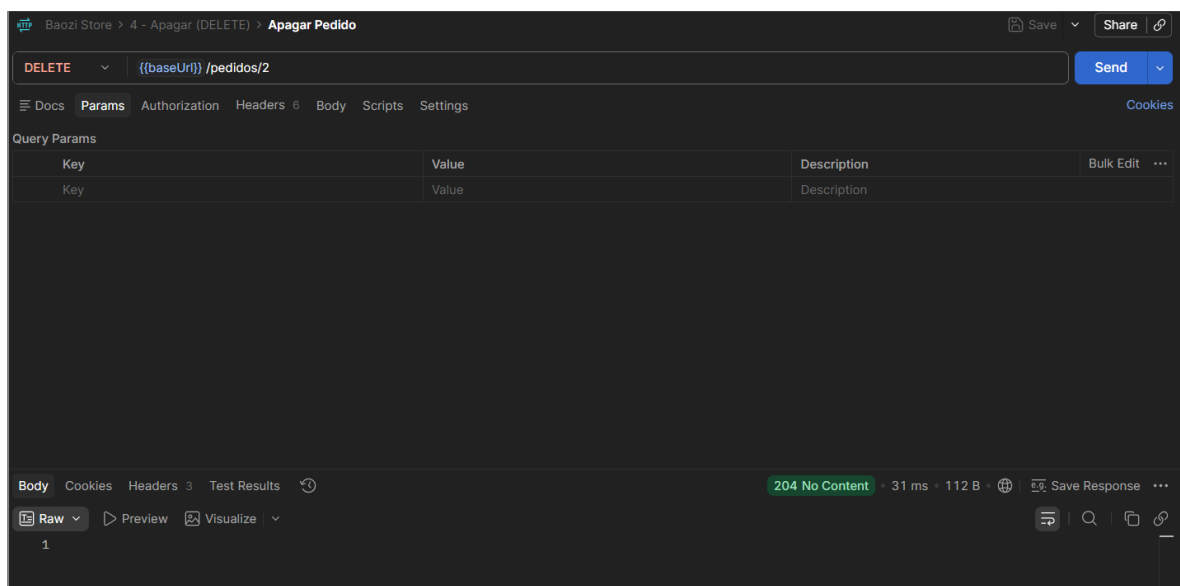


Figura 11 – DELETE /pedidos/2: exclusão do pedido extra (204 No Content)

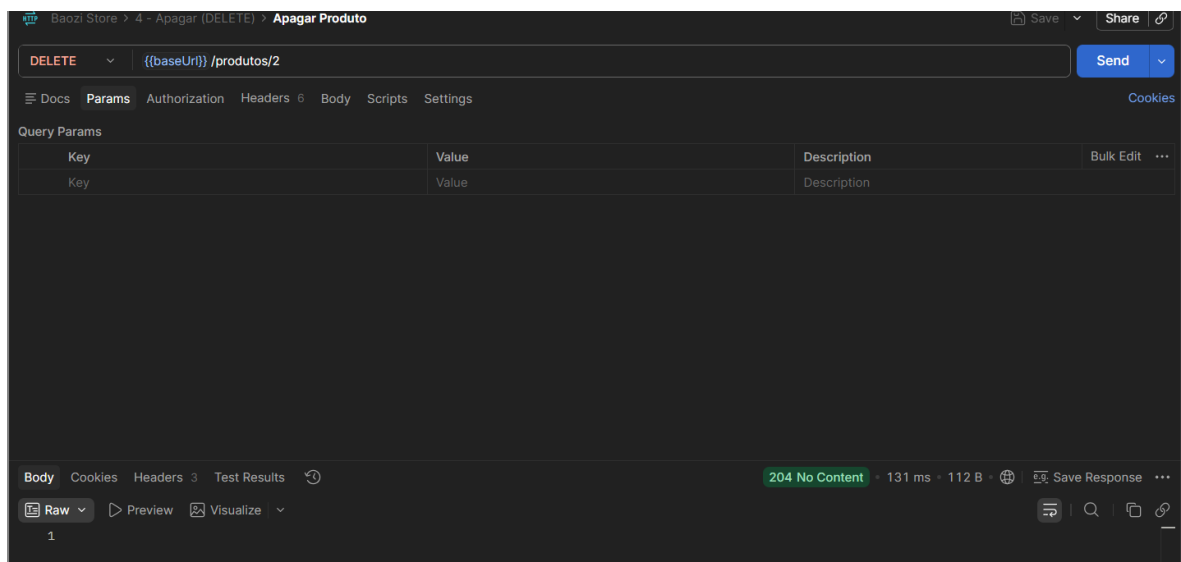


Figura 12 – DELETE /produtos/2: exclusão do produto extra (204 No Content)

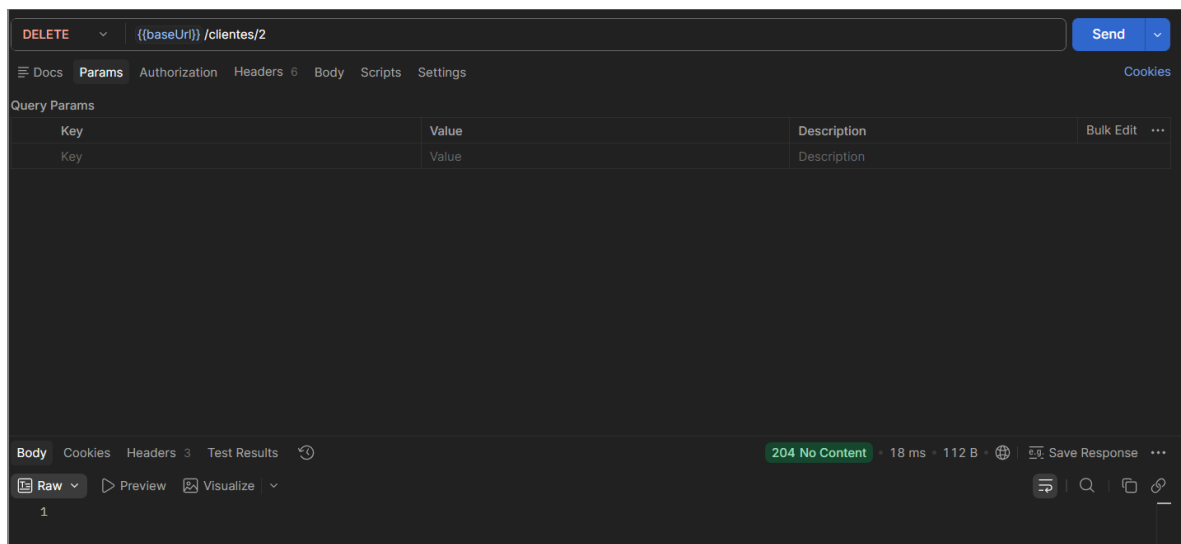


Figura 13 – DELETE /clientes/2: exclusão do cliente extra (204 No Content)

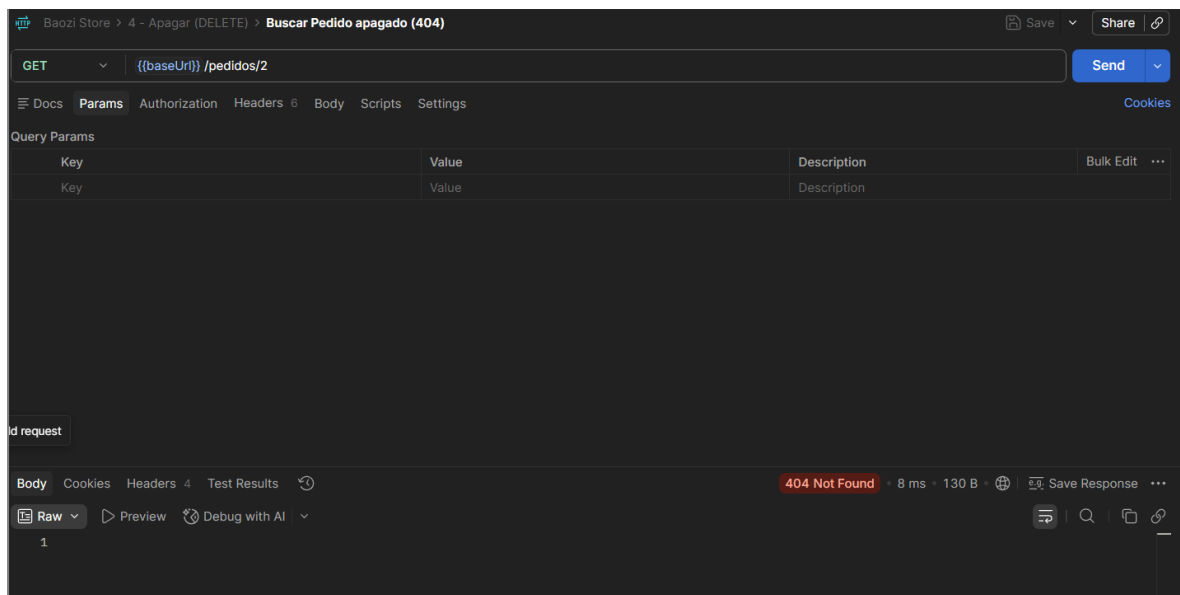


Figura 14 – GET /pedidos/2: consulta do pedido excluído (404 Not Found)

4.5 Atualização (PUT – opcional)

Nos testes de atualização, alguns valores foram alterados para demonstrar o funcionamento do PUT.

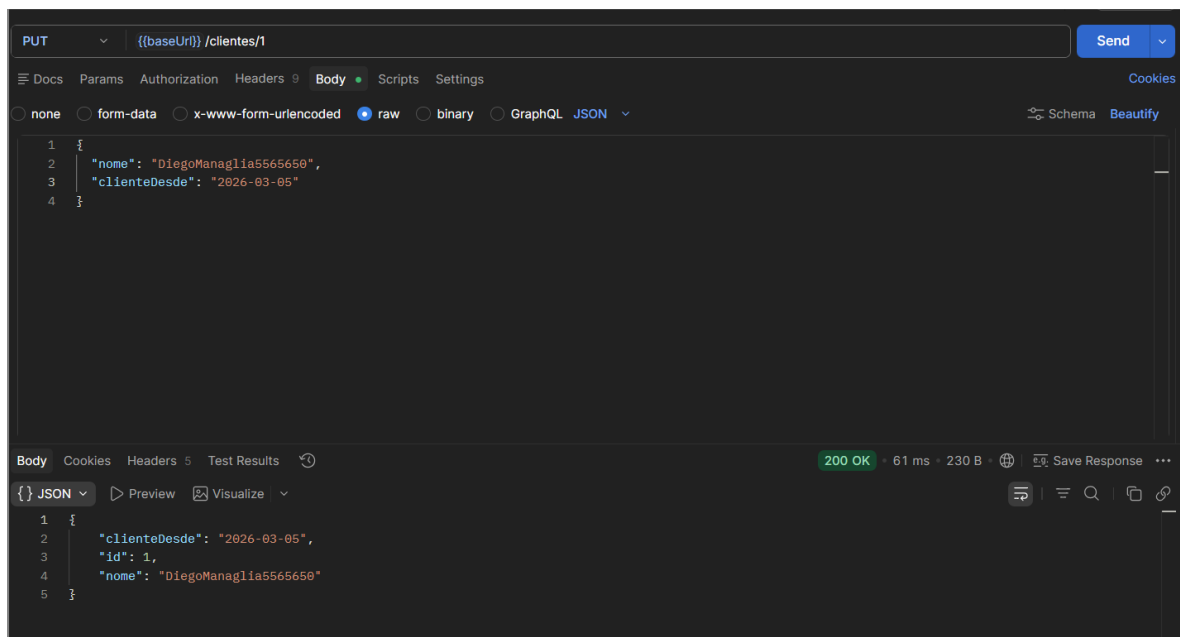


Figura 15 – PUT /clientes/1: atualização da data de cadastro do cliente (200 OK)

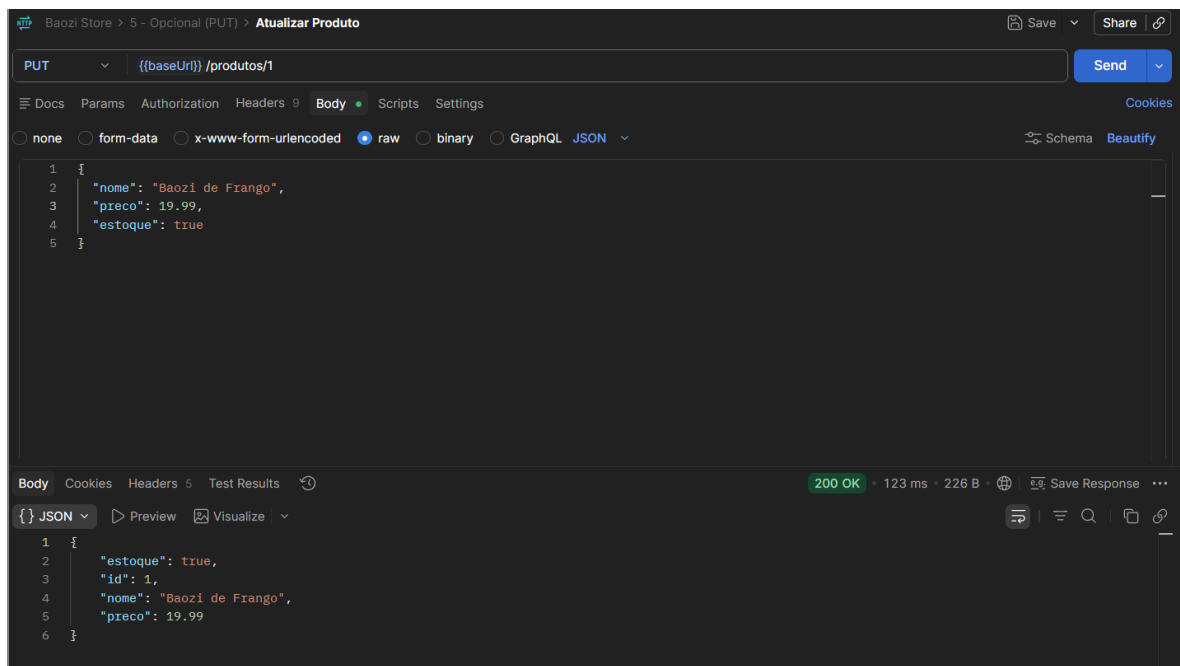


Figura 16 – PUT /produtos/1: atualização do preço do produto para R\$ 19,99 (200 OK)

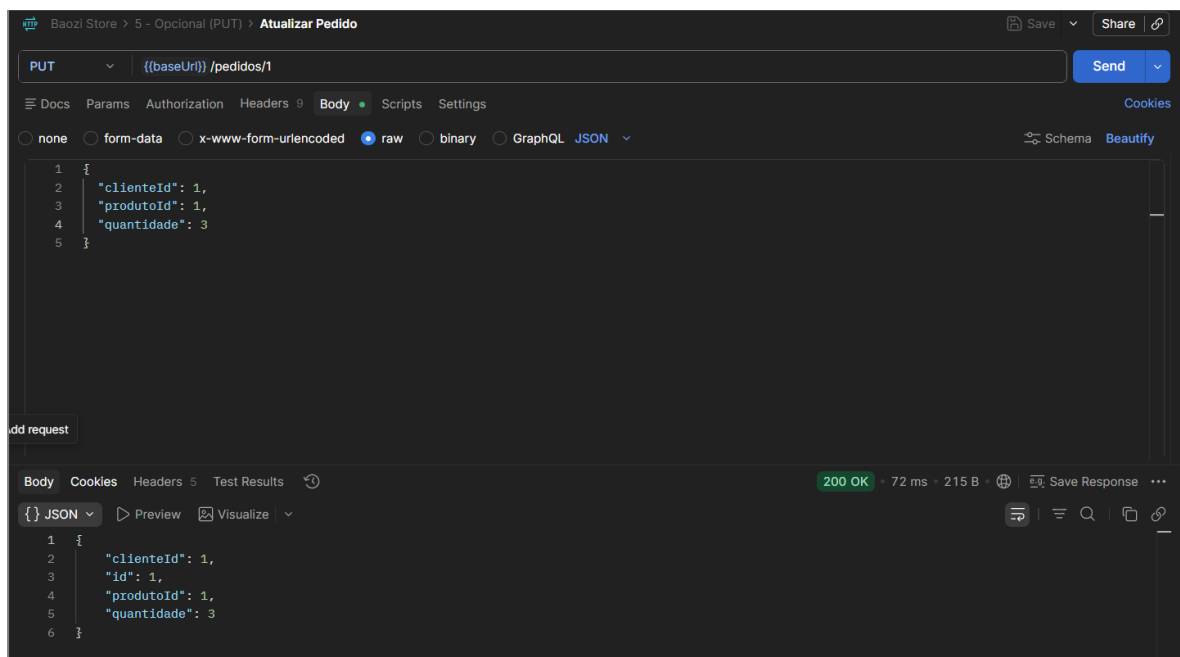


Figura 17 – PUT /pedidos/1: atualização da quantidade do pedido para 3 unidades (200 OK)

5. Link do repositório (GitHub ou similar) contendo o projeto(código fonte)

O código-fonte completo do projeto está disponível no GitHub:

<https://github.com/diegomanaglia/uninter-desenvolvimento-backend>

As instruções para executar o projeto estão no arquivo README.md do repositório. A coleção do Postman usada nos testes está em `postman/baozi-store.postman_collection.json`.